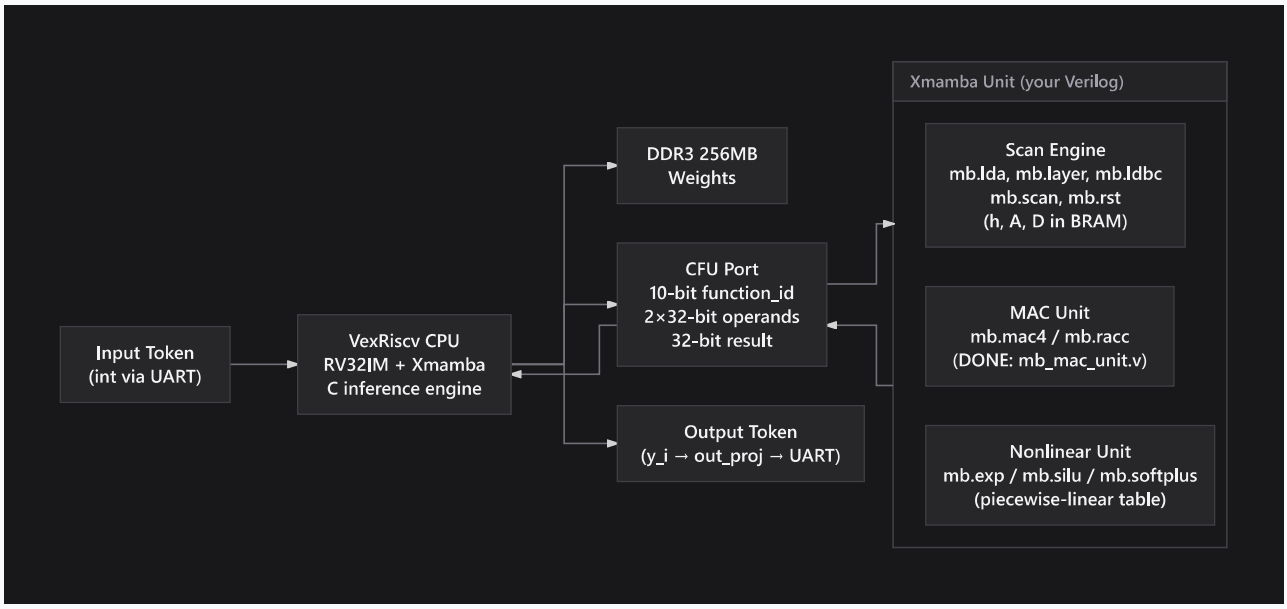
Xmamba: Running a Token Through the Entire Pipeline on CFU Playground

> IMPORTANT
>
> **You are Member A (Hardware)**. This plan is your complete build guide — from bare RTL to a single token flowing end-to-end through the custom ISA RISC-V core on CFU Playground.

What You Are Building



The **CFU Playground** builds a LiteX SoC around VexRiscv, gives you the CFU port in Verilog, and lets you run the entire system in **Renode/Verilator simulation** before the FPGA board arrives.

The One-Token Data Flow (What Happens Inside)

For every token, the C engine calls these operations in order. **Bold** = your custom instructions:

Step	Mamba Operation	Instructions Called
1	RMSNorm input (length d)	Pure software (div/sqrt in C)
2	in_proj: matrix-vector → u, z (length E·d each)	**mb.mac4** × (E·d·d/4), **mb.racc** × E·d
3	conv1d: depth-wise 4-tap, then SiLU(u)	**mb.mac4**, **mb.racc**, **mb.silu**
4	x_proj → B, C (len N each); dt_proj → Δ (softplus)	**mb.mac4**, **mb.racc**, **mb.ldbc** × N, **mb.softplus** × E·d
5	Selective scan: update h, produce y	**mb.scan** × E·d (per token, per layer)
6	y × SiLU(z); out_proj → d	**mb.silu** × E·d, **mb.mac4**, **mb.racc**
7	Add residual; next layer or decode	Software addition

Repository Layout (create this now)

```
xmamba/
  hw/
    rtl/
      mb_mac_unit.v        ← DONE (from mbmac_pkg)
      cfu_top.v            ← DONE (wraps mac unit; add NL + SCAN routing later)
      mb_nl_unit.v         ← Stage 2: nonlinear (exp/silu/softplus)
      mb_scan_engine.v     ← Stage 3: selective scan state machine
    tb/
      tb_mb_mac.sv         ← DONE (from mbmac_pkg)
      tb_mb_nl.sv          ← Stage 2
      tb_mb_scan.sv        ← Stage 3
    soc/
      build_cfu.sh         ← DONE (from mbmac_pkg)
      proj_menu.cc         ← DONE (add more test cases per stage)
  sw/
    xmamba.h               ← intrinsics for every instruction
    mamba_infer.c          ← single-token C inference engine
    mamba_infer_xmamba.c   ← accelerated version
  model/
    weights.bin            ← from Member C's .npy export
    luts.h                 ← gen_luts.py output (exp/silu/softplus tables)
  test/
    run_all.sh
    compare.py
  results/
    results.csv
```

Step-by-Step Build Plan

——— STAGE 0: Environment & Hello CFU (Now → Nov 8) ———

Step 0.1 — Install CFU Playground (Ubuntu/WSL)

```
bash
# In WSL/Ubuntu
git clone https://github.com/google/CFU-Playground.git ~/CFU-Playground
cd ~/CFU-Playground
source setup.sh              # installs LiteX, toolchain, Renode
```

> NOTE
>
> CFU Playground requires Ubuntu LTS. On Windows, use WSL2.
> If `setup.sh` fails, pin Ubuntu 22.04 in a VM — see Milestones.md Risk table.

Step 0.2 — Run the Template Project in Simulation

```
bash
cd ~/CFU-Playground/proj/proj_template_v
make renode          # builds Verilator model of the CFU + Renode SoC, runs it
# You should see the CFU Playground menu over a virtual UART
```

Step 0.3 — Install Your MAC Unit as a CFU Project

```
bash
# From your repo root:
cd ~/CFU-Playground/proj
cp -r proj_template_v mb_cfu

# Run the install script from mbmac_pkg:
bash /path/to/mbmac_pkg/soc/build_cfu.sh ~/CFU-Playground/proj/mb_cfu
# This writes cfu.v (mb_mac_unit + cfu_top) and proj_menu.cc

cd mb_cfu
make renode
# In the Renode console: press 3 (Project Menu), then d / r / b to run tests
```

Done when: `d` prints `ALL PASSED`, `r` prints `ALL PASSED`, `b` prints `0/500 wrong`.

—— STAGE 1: Verify MAC Unit RTL (Nov 9–20) ——

Step 1.1 — Run the Python Reference Tests

```
bash
cd /path/to/mbmac_pkg
python3 -m unittest discover -s tests -v
# All 6 unit tests should pass
```

Step 1.2 — Run the Full Vector Suite (Icarus Verilog)

```
bash
sudo apt install iverilog    # once
python3 -m mbmac.verify --quick       # ~80k commands, quick smoke test
python3 -m mbmac.verify               # full 347,748 commands
# Expected: "PASS mb_mac OUT_BITS=8 records=347748 ..."
```

Step 1.3 — Verilator Lint

```
bash
verilator --lint-only -Wall -Wno-DECLFILENAME \
  rtl/mb_mac_unit.v rtl/cfu_top.v --top-module Cfu
```

Step 1.4 — Vivado Out-of-Context Synthesis

Run this even before the board arrives to get LUT/DSP/timing numbers:

```
tcl
# synth.tcl
read_verilog mb_mac_unit.v
synth_design -top mb_mac_unit -part xc7a100tcsg324-1
report_utilization
report_timing_summary
```

```
bash
vivado -mode batch -source synth.tcl
```

Record: LUTs, DSPs, worst slack at your target clock (e.g. 50 MHz).

> **TIP**
>
> If the four 8×8 multipliers map to LUTs instead of DSPs, add `(* use_dsp = "yes" *)` to `m0..m3` in `mb_mac_unit.v` and re-synthesize.

—— STAGE 2: Nonlinear Unit — mb.exp / mb.silu / mb.softplus (Nov 23–Dec 11) ——

This is the second sub-unit of your `cfu_top.v`. It handles activations needed for SiLU and the scan's softplus(Δ).

Step 2.1 — Get the Lookup Tables from Member C

Member C's `gen_luts.py` produces a C header with the piecewise-linear segment data:

```
c
// luts.h (generated)
// 16 segments for exp (input range [-8.5, 0])
// 32 segments for silu and softplus
typedef struct { int32_t x0, slope, offset; } Segment;
extern const Segment exp_lut[16];
extern const Segment silu_lut[32];
extern const Segment softplus_lut[32];
```

The input to each function is a Q4.12 fixed-point value (16-bit signed, 12 fractional bits).

Step 2.2 — Write `mb_nl_unit.v`

Architecture:

- **ROM**: the LUT stored in block RAM (one 36-bit entry = slope + offset per segment).
- **Index**: top bits of `rs1` select the segment → ROM address.
- **Interpolation**: `result = offset + slope * (rs1 & frac_mask)` — one multiplier, one adder.
- **Latency**: 2 cycles (ROM read + multiply).

Instruction encoding (funct3 = 1):

funct7	Instruction	Effect
0	mb.exp	rd = exp(rs1) in Q4.12
1	mb.silu	rd = rs1 * sigmoid(rs1) in Q4.12
2	mb.softplus	rd = log(1 + exp(rs1)) in Q4.12

Step 2.3 — Update `cfu_top.v`

```
verilog
// Route by funct3
wire [2:0] f3 = cmd_payload_function_id[2:0];

// Instantiate both units, arbitrate cmd_valid/ready
assign mac_cmd_valid = cmd_valid & (f3 == 3'd0);
assign nl_cmd_valid  = cmd_valid & (f3 == 3'd1);

assign cmd_ready = (f3 == 3'd0) ? mac_cmd_ready : nl_cmd_ready;
assign rsp_valid = mac_rsp_valid | nl_rsp_valid;
assign rsp_payload_outputs_0 = mac_rsp_valid ? mac_rsp_data : nl_rsp_data;
```

Step 2.4 — Write `xmamba.h` Intrinsics

```
c
// xmamba.h
#define CFU_INSN(f7, f3, rd, rs1, rs2) \
    asm volatile (".insn r CUSTOM_0, " #f3 ", " #f7 ", %0, %1, %2" \
                  : "=r"(rd) : "r"(rs1), "r"(rs2))

static inline void     mb_mac4(uint32_t a, uint32_t b)   { uint32_t _; CFU_INSN(0,0,_,a,b);
}
static inline int32_t  mb_racc(uint32_t shift)            { int32_t r;
CFU_INSN(1,0,r,shift,0); return r; }
static inline int32_t  mb_exp(int32_t x)                  { int32_t r; CFU_INSN(0,1,r,x,0);
return r; }
static inline int32_t  mb_silu(int32_t x)                 { int32_t r; CFU_INSN(1,1,r,x,0);
return r; }
static inline int32_t  mb_softplus(int32_t x)             { int32_t r; CFU_INSN(2,1,r,x,0);
return r; }
```

> **NOTE**
>
> CFU Playground also provides `cfu_op0(funct7, rs1, rs2)` as a convenience macro for funct3=0. Use `.insn` directly for funct3 ≠ 0 (nonlinear and scan instructions).

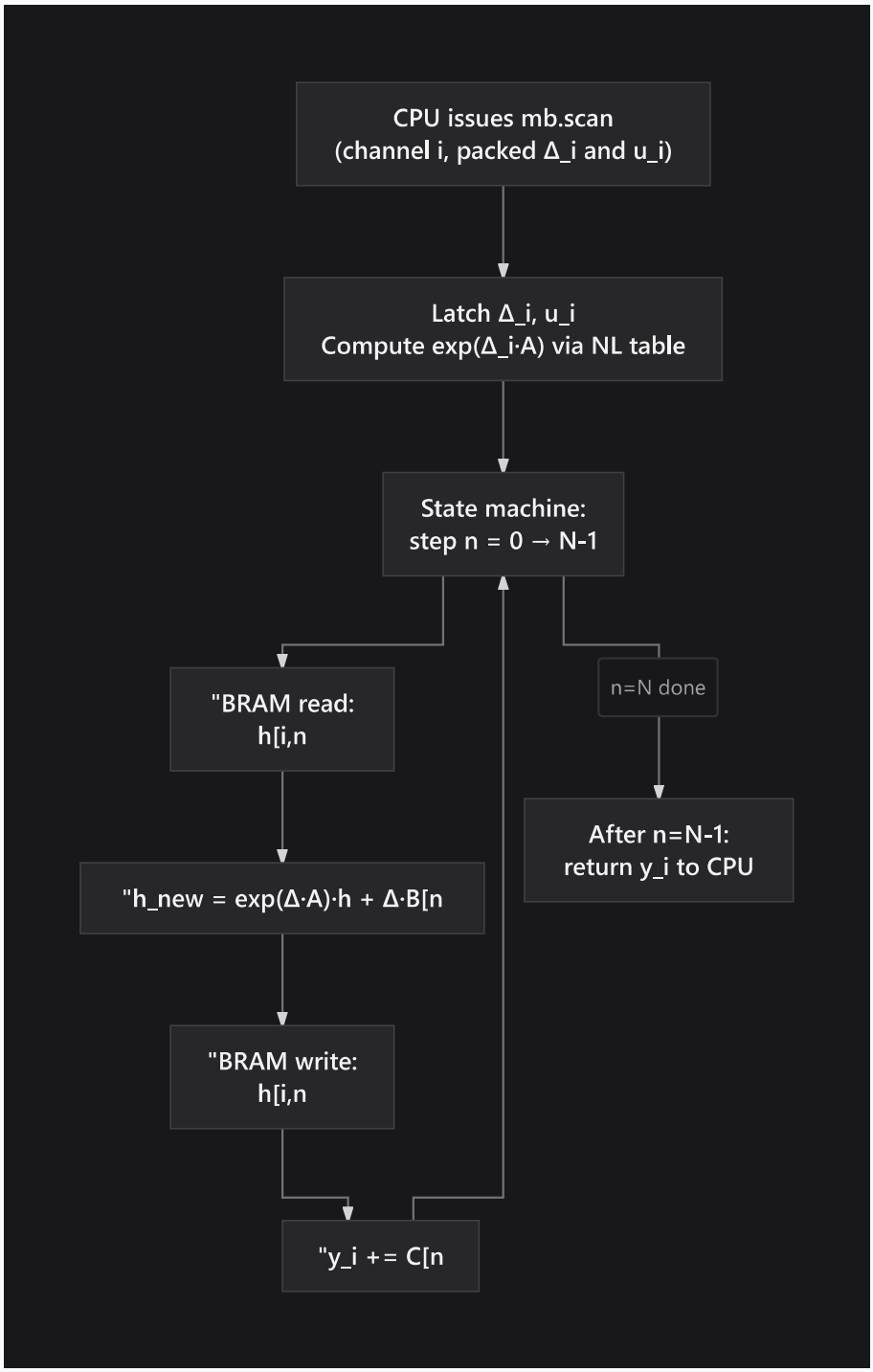
—— STAGE 3: Scan Engine — mb.lda, mb.layer, mb.ldbc, mb.scan, mb.rst (Jan 4–22) ——

This is the heart of Mamba. The scan engine lives inside the FPGA and holds the hidden state `h` (shape: layers × E·d × N × 4 bytes) in block RAM.

Scan Engine Instruction Table (funct3 = 2)

funct7	Instruction	rs1	rs2	Effect
0	mb.rst	—	—	Clear all h to 0
1	mb.layer	layer_idx	—	Select active layer
2	mb.lda	addr (i*N+n)	A_in packed Q16	Write A[i,n] or D[i] into BRAM
3	mb.ldbc	n	B_n|C_n packed	Write B[n] (bits[31:16]) and C[n] (bits[15:0]) for current token
4	mb.scan	channel i	Δ_i|u_i packed	Run the scan for all N states of channel i, return y_i

Scan Engine Microarchitecture



Latency of mb.scan: approximately N + 3 pipeline fill cycles. With N=16, that is ~19 cycles per channel.

BRAM Layout

```
State BRAM (dual-port, read/write same port A, read port B):
  Addr = layer_idx * (E·d * N) + channel_i * N + state_n
  Width = 32 bits (Q16.16 fixed point for h)

Constant BRAM (read-only):
  A[layer][i][n] at addr = layer * E·d * N + i * N + n
  D[layer][i]    at addr = LAYERS * E·d * N + layer * E·d + i
```

Step 3.1 — Write `mb_scan_engine.v`

Key signals and state machine:

```
verilog
// State registers
reg [4:0]  n_r;          // current state index (0..N-1)
reg [31:0] delta_r, u_r; // latched operands
reg [31:0] yi_acc;       // accumulates C[n]*h[i,n]
reg        busy;

// BRAM A: h state (dual-port)
// BRAM B: A constants (ROM)
// Shared NL unit call for exp(delta * A[i,n])

always @(posedge clk) begin
    if (accept && is_scan) begin
        delta_r <= rs1[31:16]; // Δ_i (Q8.8 or agreed format)
        u_r     <= rs1[15:0];  // u_i
        channel <= rs2;
        n_r     <= 0;
        yi_acc  <= 0;
        busy    <= 1;
    end
    if (busy) begin
        // pipeline: read h[i,n], read A[i,n], call exp via NL, update h, write back,
accumulate
        // ...
        if (n_r == N-1) begin
            busy       <= 0;
            rsp_data_r <= yi_acc + D[channel] * u_r; // final y_i
            rsp_valid_r <= 1;
        end
    end
end
```

Step 3.2 — Update `cfu_top.v` for Three Units

```
verilog
assign mac_cmd_valid  = cmd_valid & (f3 == 3'd0);
assign nl_cmd_valid   = cmd_valid & (f3 == 3'd1);
assign scan_cmd_valid = cmd_valid & (f3 == 3'd2);

assign cmd_ready = (f3==0) ? mac_rdy : (f3==1) ? nl_rdy : scan_rdy;
assign rsp_valid = mac_rsp_valid | nl_rsp_valid | scan_rsp_valid;
assign rsp_payload_outputs_0 =
    mac_rsp_valid  ? mac_data  :
    nl_rsp_valid   ? nl_data   : scan_data;
```

—— STAGE 4: C Inference Engine (runs on VexRiscv, calls CFU) ——

This is the software side that drives your hardware. Member B writes most of this, but you need to understand it to debug mismatches.

`mamba_infer.c` — One Token Through One Layer

```
c
#include "xmamba.h"
#include "luts.h"   // LUT data (from gen_luts.py)

// Model dimensions (fill from model card)
#define D_MODEL  128
#define EXPAND   2
#define D_INNER  (D_MODEL * EXPAND)  // E·d
#define D_STATE  16                  // N
#define D_CONV   4
#define DT_RANK  8

// All weights are int8, pre-loaded into DDR by the startup code
extern int8_t W_in_proj[D_INNER*2][D_MODEL];  // in_proj
extern int8_t W_out_proj[D_MODEL][D_INNER];   // out_proj
// ... etc.

// ---- mb.mac4 dot-product helper ----
// Compute dot(a, b, len) of int8 vectors, return int8 via mb.racc(shift)
int8_t dot8(const int8_t *a, const int8_t *b, int len, int shift) {
    mb_racc(0);  // clear accumulator first
    for (int i = 0; i < len; i += 4)
        mb_mac4(*(uint32_t*)(a+i), *(uint32_t*)(b+i));
    return (int8_t)mb_racc(shift);
}

// ---- One Mamba layer, one token ----
void mamba_layer(int layer, const int8_t *x_in, int8_t *x_out) {
    // 1. RMSNorm (software)
    int8_t x_norm[D_MODEL];
    rms_norm(x_in, x_norm, D_MODEL);  // software implementation

    // 2. in_proj: [D_INNER*2, D_MODEL] × x_norm → u_gate (packed u and z)
    int8_t u[D_INNER], z[D_INNER];
    for (int i = 0; i < D_INNER; i++)
        u[i] = dot8(W_in_proj[i], x_norm, D_MODEL, SCALE_SHIFT_IN);
    for (int i = 0; i < D_INNER; i++)
        z[i] = dot8(W_in_proj[D_INNER+i], x_norm, D_MODEL, SCALE_SHIFT_IN);

    // 3. conv1d: depth-wise 4-tap on u, then SiLU
    static int8_t conv_buf[D_INNER][D_CONV];  // ring buffer
    int8_t xu[D_INNER];
    for (int i = 0; i < D_INNER; i++) {
        // shift ring buffer, insert u[i]
```

```
        memmove(conv_buf[i], conv_buf[i]+1, (D_CONV-1)*sizeof(int8_t));
        conv_buf[i][D_CONV-1] = u[i];
        // 4-tap conv
        mb_racc(0);
        mb_mac4(*(uint32_t*)conv_buf[i], *(uint32_t*)W_conv[i]);
        int8_t v = (int8_t)mb_racc(SCALE_SHIFT_CONV);
        // SiLU via custom instruction
        xu[i] = (int8_t)mb_silu(v << FRAC_BITS);  // convert to Q4.12
    }

    // 4. x_proj → B, C, dt; dt_proj → Δ (softplus)
    int16_t B[D_STATE], C[D_STATE];
    int8_t  delta[D_INNER];
    for (int n = 0; n < D_STATE; n++) {
        B[n] = dot16(W_x_proj_B[n], xu, D_INNER, SCALE_SHIFT_B);
        C[n] = dot16(W_x_proj_C[n], xu, D_INNER, SCALE_SHIFT_C);
    }
    for (int i = 0; i < D_INNER; i++) {
        int8_t dt_raw = dot8(W_dt[i], xu, DT_RANK, SCALE_SHIFT_DT);
        delta[i] = (int8_t)mb_softplus(dt_raw << FRAC_BITS);
    }

    // 5. Load B and C into the scan engine, run scan for all channels
    mb_layer(layer);  // select this layer's BRAM bank
    for (int n = 0; n < D_STATE; n++)
        mb_ldbc(n, ((uint32_t)(uint16_t)B[n] << 16) | (uint16_t)C[n]);

    int8_t y[D_INNER];
    for (int i = 0; i < D_INNER; i++) {
        uint32_t delta_u = ((uint32_t)(uint8_t)delta[i] << 16) | (uint8_t)xu[i];
        y[i] = (int8_t)mb_scan(i, delta_u);  // returns y_i
    }

    // 6. y * SiLU(z), then out_proj
    int8_t yz[D_INNER];
    for (int i = 0; i < D_INNER; i++) {
        int32_t sz = mb_silu(z[i] << FRAC_BITS);
        yz[i] = sat8((y[i] * sz) >> FRAC_BITS);
    }
    for (int i = 0; i < D_MODEL; i++)
        x_out[i] = dot8(W_out_proj[i], yz, D_INNER, SCALE_SHIFT_OUT);

    // 7. Add residual
    for (int i = 0; i < D_MODEL; i++)
        x_out[i] = sat8((int32_t)x_out[i] + x_in[i]);
}
```

—— STAGE 5: Full Token Pipeline — End-to-End Test ——

Step 5.1 — Startup: Load Constants into Scan Engine

At boot (once per model load):

```
c
void mamba_init(void) {
    mb_rst();  // clear all hidden state h
    // For each layer, for each channel i, for each state n:
    for (int l = 0; l < N_LAYERS; l++) {
        mb_layer(l);
        for (int i = 0; i < D_INNER; i++) {
            for (int n = 0; n < D_STATE; n++) {
                uint32_t addr = i * D_STATE + n;
                mb_lda(addr, A_constants[l][i][n]);  // write A into scan BRAM
            }
            mb_lda(D_INNER * D_STATE + i, D_constants[l][i]);  // write D
        }
    }
}
```

Step 5.2 — Token Generation Loop

```
c
void generate(const char *prompt, int max_tokens) {
    int8_t h[D_MODEL];
    // ... embed prompt token ...
    mamba_init();

    for (int t = 0; t < max_tokens; t++) {
        // Run all layers
        for (int l = 0; l < N_LAYERS; l++)
            mamba_layer(l, h, h);   // in-place residual stream

        // Decode: linear projection to vocab, argmax (software)
        int next_token = lm_head(h);
        print_token(next_token);
        if (next_token == EOS_TOKEN) break;

        // Embed next token as input
        embed(next_token, h);
    }
}
```

Step 5.3 — Run in Renode Simulation

```
bash
cd ~/CFU-Playground/proj/mb_cfu
make renode
# In Renode console:
# Type 3 (Project Menu)
# Type 't' (token test) — add this to proj_menu.cc
```

Add a token test to `proj_menu.cc`:

```
cpp
void do_token_test(void) {
    puts("\nRunning single token through full Mamba pipeline\n");
    mamba_init();
    int8_t x[D_MODEL] = { /* embed token 42 */ };
    int8_t y[D_MODEL];

    uint32_t cyc_start = measure_cycles();
    for (int l = 0; l < N_LAYERS; l++)
        mamba_layer(l, x, y);
    uint32_t cyc_end = measure_cycles();

    printf("Cycles per token: %lu\n", (unsigned long)(cyc_end - cyc_start));
    // Compare y[] against expected[] from test vectors
    compare_output(y, expected_output, D_MODEL);
}
```

—— STAGE 6: FPGA Bring-Up (Jan 4–Feb 12) ——

Sequential steps — each one adds exactly one new thing:

Step	Command	What Can Break
1. Stock bitstream + UART hello	`make prog` (stock SoC)	Board connection, USB UART driver
2. DDR test	`make prog` with DDR test firmware	Clock/calibration, LiteDRAM
3. Baseline C engine (no CFU)	Compile with `-DBASELINE`, flash	Software bugs visible
4. MAC unit only	`make bitstream` with `mb_mac_unit` CFU	DSP mapping, timing at 50MHz
5. + Nonlinear unit	Add `mb_nl_unit.v` to `cfu_top.v`, rebuild	BRAM for LUT, interpolation
6. + Scan engine	Add `mb_scan_engine.v`, rebuild	BRAM size, scan state machine
7. Full model	Flash full `mamba_infer_xmamba.c`	End-to-end correctness

Build the bitstream:

```
bash
cd ~/CFU-Playground/proj/mb_cfu
make bitstream    # Vivado synthesis + implementation
make prog         # Flash to Arty A7-100T
make terminal     # Open UART terminal
```

What to Build — Summary Checklist

Hardware Files (Your Deliverables — Member A)

File	Status	Deadline
	✅ DONE	Nov 8
(MAC only)	✅ DONE	Nov 8
`mb_nl_unit.v` (exp/silu/softplus table lookup)	☐ TODO	Dec 11
`cfu_top.v` updated (MAC + NL routing)	☐ TODO	Dec 11
`mb_scan_engine.v` (selective scan state machine)	☐ TODO	Jan 15
`cfu_top.v` final (MAC + NL + Scan)	☐ TODO	Jan 18
Integrate into SoC: one full token on simulated SoC	☐ TODO	Jan 22
Weekly Vivado synthesis reports	☐ TODO	Ongoing
FPGA bring-up on Arty A7-100T	☐ TODO	Feb 12

What to Hand Off (Interface Contracts)

You hand to	What	When
Member B	CPU–CFU interface summary (handshake signals, latency per instruction)	Nov 8
Member B	ISA v0 review: implementability, DSP/BRAM cost, critical path estimate	Nov 20
Member B	Microarchitecture note: block diagram, scan state machine, BRAM layout	Dec 11
Member D	Weekly Vivado utilization/timing numbers for results.csv	Every Friday

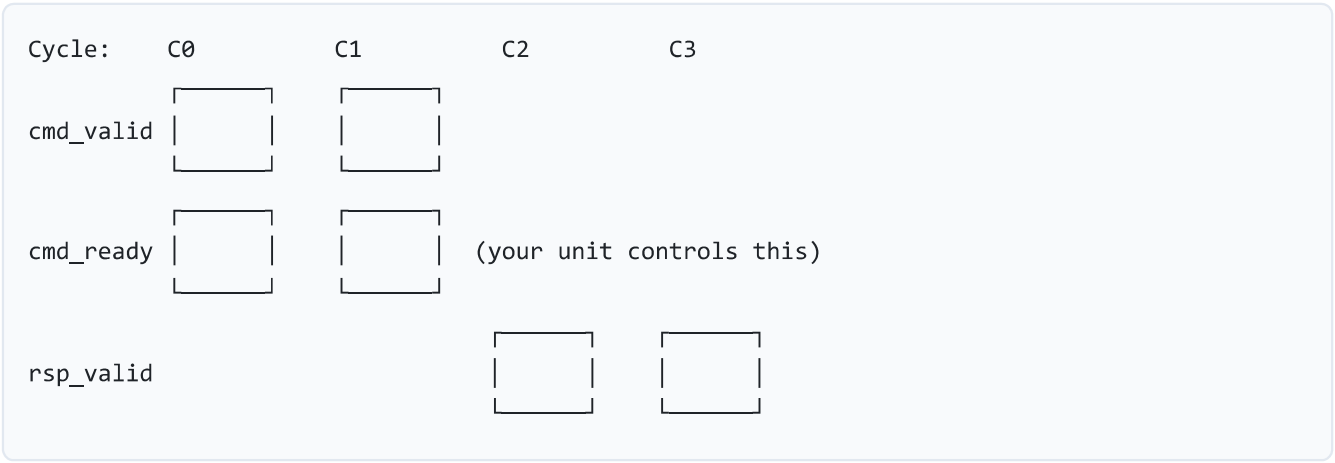
Instruction Encoding Reference Card

All instructions use **custom-0 opcode** (`0001011`), R-type format.

Instruction	funct3	funct7	rs1	rs2	rd	C call
mb.mac4	0	0	4×int8 activation	4×int8 weight	0	`cfu_op0(0, rs1, rs2)`
mb.racc	0	1	shift[4:0]	—	sat_int8	`cfu_op0(1, shift, 0)`
mb.exp	1	0	Q4.12 input	—	Q4.12 result	`.insn r CUSTOM_0,1,0,rd,rs1,x0`
mb.silu	1	1	Q4.12 input	—	Q4.12 result	`.insn r CUSTOM_0,1,1,rd,rs1,x0`
mb.softplus	1	2	Q4.12 input	—	Q4.12 result	`.insn r CUSTOM_0,1,2,rd,rs1,x0`
mb.rst	2	0	—	—	—	`.insn r CUSTOM_0,2,0,x0,x0,x0`
mb.layer	2	1	layer_idx	—	—	`.insn r CUSTOM_0,2,1,x0,rs1,x0`
mb.lda	2	2	addr	A or D value	—	`.insn r CUSTOM_0,2,2,x0,rs1,rs2`
mb.ldbc	2	3	n	B|C packed	—	`.insn r CUSTOM_0,2,3,x0,rs1,rs2`
mb.scan	2	4	Δ|u packed	channel i	y_i	`.insn r CUSTOM_0,2,4,rd,rs1,rs2`

CFU Handshake — Critical Timing Rules

From the CFU Playground interface spec:



Rules you must follow:

1. Operands (`cmd_payload_inputs_0/1`) are valid for **one cycle only** — latch them immediately.
2. If your unit never asserts `rsp_valid`, the CPU hangs forever. **Always respond.**
3. `cmd_ready` must go low while a response is pending or multi-cycle work is in progress.
4. `mb.scan` takes ~N+3 cycles — the CPU stalls during this time (normal behavior).

BRAM Budget (XC7A100T has 4,860 Kbit total)

Use	Size	Formula
NL unit LUT ROM	~2 KB	3 functions × 32 segments × 8 bytes/segment
Scan h state	layers × E·d × N × 4 B	e.g. 6 × 256 × 16 × 4 = **96 KB**
Scan A/D constants	layers × (E·d × N + E·d) × 4 B	e.g. 6 × (4096+256) × 4 = **105 KB**
Total Xmamba BRAM	**~203 KB**	<< 607 KB available ✅

Key Files in the Repository

File	What it is
	MAC unit RTL (mb.mac4 + mb.racc)
	CFU Playground top module
	Golden Python reference
	Test vector generator
	SystemVerilog testbench
	On-SoC C test harness
	Installs RTL into CFU Playground project
	Full project plan (all members)